

# Rechnergestützte Numerik und Simulation

(am Beispiel von Matlab/Simulink)

## Implementierung und Matlab-Praxis 1:

### *Matlab-Einführung*

# Inhalte

## 1. Grundlagen

- 1.1 Was ist Matlab?
- 1.2 Benutzeroberfläche
- 1.3 Der Matlab-Workspace
- 1.4 Vektoren und Matrizen
- 1.5 M-Skripte und M-Editor
- 1.6 Kontrollstrukturen
- 1.7 Grafik

# 1. Einführung – 1.1 Was ist Matlab?

## Eigenschaften von Matlab:

- Verwendung der Interpretersprache „M“ in Kommandozeilen und über so genannte m-Skripten (auch kombiniert).
- Umfangreicher Befehlssatz für numerische Aufgaben und Simulation
- Effiziente Bearbeitung von Matrizen und Vektoren durch einen weitgehend „vektorisierten“ Befehlssatz.
- Umfangreicher Befehlssatz für graphische Darstellung Simulations- und Messdaten.
- Direkt Kopplung zum Simulationstool „Simulink“ (häufig wird auch nur von „Matlab/Simulink“ gesprochen), d. h. Simulationssteuerung, Modellparametrierung und Modellkonfiguration sowie Postprozess für Simulationsdaten kann durch m-Skripte erfolgen.



# 1. Einführung – 1.1 Was ist Matlab?

## Literatur zu MatLab

- A. Angermann, M. Beuschel, M. Rau, U. Wohlfarth  
**MATLAB - SIMULINK - STATEFLOW**  
**Grundlagen, Toolboxes, Beispiele**  
5., überarbeitete Auflage,  
[Oldenbourg Verlag](#), München 2007.  
Reihe: Oldenbourg Lehrbücher für Ingenieure  
  
mit CD-ROM  
ISBN 978-3-486-58272-7  
**Preis:** € 34,80
- Wolfgang Schweizer:  
**MATLAB kompakt**  
4. Auflage  
[Oldenbourg Verlag](#), München 2009.  
  
ISBN 978-3-486-59193-4



# 1. Einführung – 1.1 Was ist Matlab?

## Erweiterungen:

- Simulink → Grafische Simulationsumgebung (s. o.)
  
- Diverse sogenannte Toolboxen (als Packages für Matlab oder Simulink), z. B.:
  - Control-System-Toolbox → Matlab-Befehlssatzerweiterung für Regelungstechnik.
  - Signal-Processing-Toolbox → Matlab-Befehlssatzerweiterung für Signalverarbeitung.
  - SimPowerSystems → Simulink-Erweiterung zur Schaltungssimulation.
  - State-Flow → Zustandsgraphen unter Simulink.
  
- Code-Generatoren
  - Matlab-Coder → c-Code-Erzeugung aus m-Skripten
  - RTW → Real-Time-Workshop (Echtzeit c-Code-Erzeugung aus Simulink)
  - HDL-Coder → VHDL-Code-Erzeugung aus Simulink



# 1. Einführung – 1.2 Benutzeroberfläche

The image shows the MATLAB R2016b - academic use interface. The main window is divided into several panes. The top pane is the Command Window, which contains a message: "New to MATLAB? See resources for [Getting Started](#)." Below this is the Command History pane, which lists the commands entered in the Command Window. The bottom pane is the Workspace Browser, which displays the variables in the workspace. The left pane is the Current Directory Browser, which shows the current directory and its contents. The top toolbar contains various icons for file operations, variable management, and code execution. The status bar at the bottom indicates "Ready".

Annotations:

- Current Directory Browser
- Current Directory
- Command Window
- Command History
- Workspace Browser

# 1. Einführung – 1.2 Benutzeroberfläche: MATLAB Hilfe

Die Matlab-Hilfe-Fenster kann über das Menü des Matlab-Windows oder über die Kommandozeilenbefehle

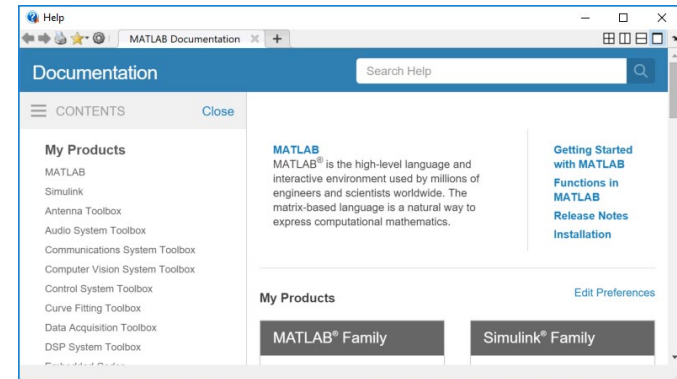
```
>> doc
```

oder

```
>>helpwin
```

aufgerufen werden.

```
>>doc <command>
```



Geht direkt zur Hilfeinformation des Befehles `<command>`.

Außerdem gibt der Befehl

```
>>help <command>
```

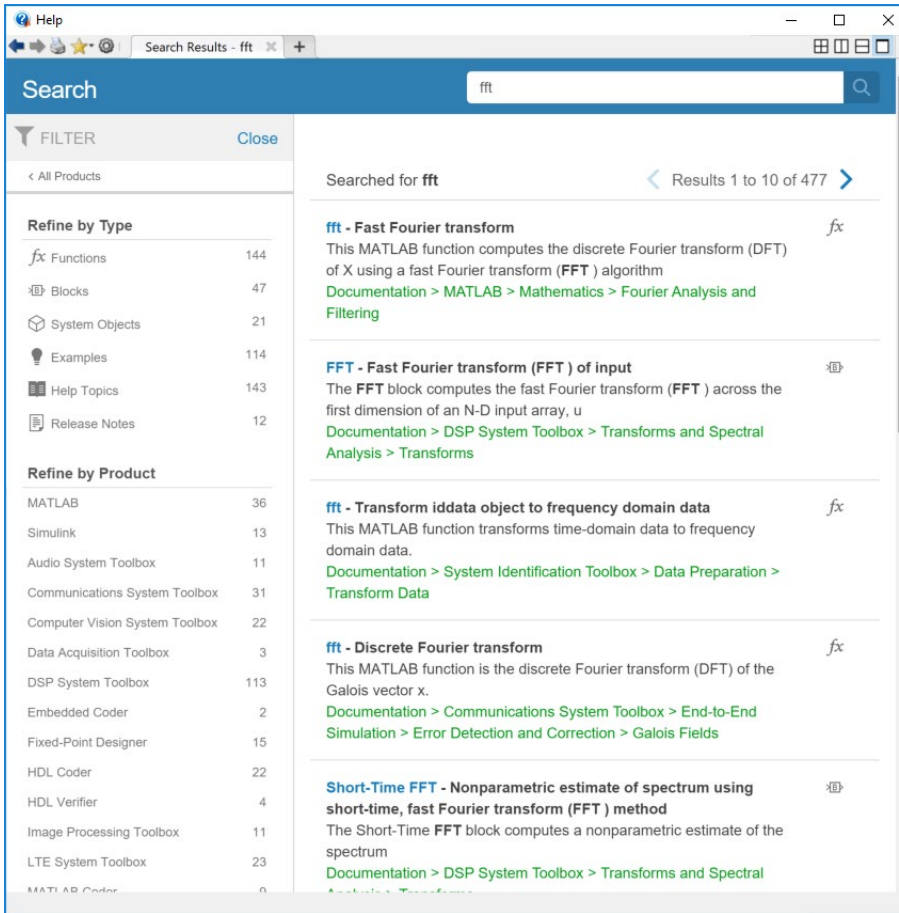
direkt im Kommandofenster eine Hilfeinformation zum Befehle `<command>` aus. Diese Hilfeinformation enthält üblicherweise auch hilfreiche Hinweise auf verwandte Befehle.

Der Befehl

```
>>lookfor <keyword>
```

Ermöglicht die Suche nach Befehlen mit dem Schlüsselwort `<keyword>` im Hilfetext.

# 1. Einführung – 1.2 Benutzeroberfläche: MATLAB Hilfe



Help

Search Results - fft

Search

fft

FILTER Close

< All Products

Refine by Type

- Functions 144
- Blocks 47
- System Objects 21
- Examples 114
- Help Topics 143
- Release Notes 12

Refine by Product

- MATLAB 36
- Simulink 13
- Audio System Toolbox 11
- Communications System Toolbox 31
- Computer Vision System Toolbox 22
- Data Acquisition Toolbox 3
- DSP System Toolbox 113
- Embedded Coder 2
- Fixed-Point Designer 15
- HDL Coder 22
- HDL Verifier 4
- Image Processing Toolbox 11
- LTE System Toolbox 23
- MATLAB Coder 0

Searched for fft Results 1 to 10 of 477

**fft - Fast Fourier transform** *fx*

This MATLAB function computes the discrete Fourier transform (DFT) of X using a fast Fourier transform (FFT) algorithm

[Documentation > MATLAB > Mathematics > Fourier Analysis and Filtering](#)

**FFT - Fast Fourier transform (FFT) of input** *fx*

The FFT block computes the fast Fourier transform (FFT) across the first dimension of an N-D input array, u

[Documentation > DSP System Toolbox > Transforms and Spectral Analysis > Transforms](#)

**fft - Transform iddata object to frequency domain data** *fx*

This MATLAB function transforms time-domain data to frequency domain data.

[Documentation > System Identification Toolbox > Data Preparation > Transform Data](#)

**fft - Discrete Fourier transform** *fx*

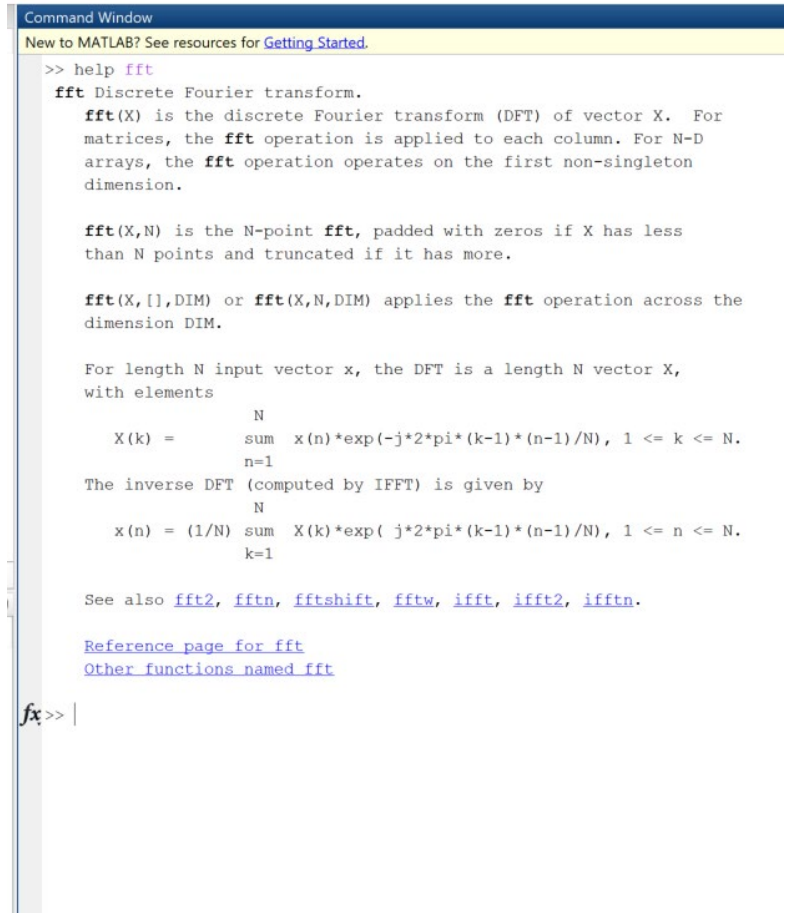
This MATLAB function is the discrete Fourier transform (DFT) of the Galois vector x.

[Documentation > Communications System Toolbox > End-to-End Simulation > Error Detection and Correction > Galois Fields](#)

**Short-Time FFT - Nonparametric estimate of spectrum using short-time, fast Fourier transform (FFT) method** *fx*

The Short-Time FFT block computes a nonparametric estimate of the spectrum

[Documentation > DSP System Toolbox > Transforms and Spectral Analysis > Transforms](#)



Command Window

New to MATLAB? See resources for [Getting Started](#).

```
>> help fft
fft Discrete Fourier transform.

fft(X) is the discrete Fourier transform (DFT) of vector X. For
matrices, the fft operation is applied to each column. For N-D
arrays, the fft operation operates on the first non-singleton
dimension.

fft(X,N) is the N-point fft, padded with zeros if X has less
than N points and truncated if it has more.

fft(X,[],DIM) or fft(X,N,DIM) applies the fft operation across the
dimension DIM.

For length N input vector x, the DFT is a length N vector X,
with elements

      N
      sum
X(k) =  x(n)*exp(-j*2*pi*(k-1)*(n-1)/N), 1 <= k <= N.
      n=1

The inverse DFT (computed by ifft) is given by

      N
      sum
x(n) = (1/N) X(k)*exp( j*2*pi*(k-1)*(n-1)/N), 1 <= n <= N.
      k=1

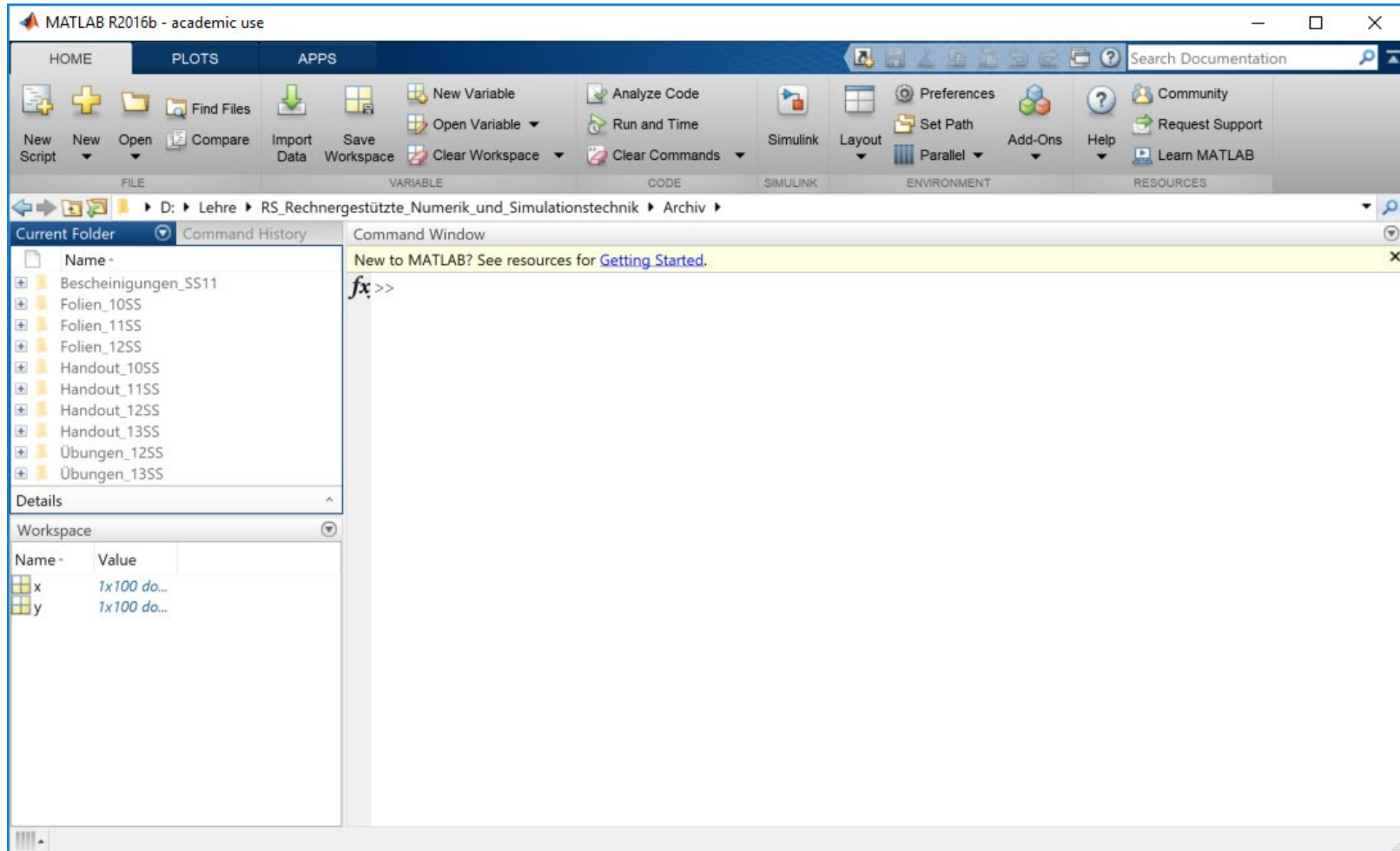
See also fft2, fftn, fftshift, fftw, ifft, ifft2, ifftn.

Reference page for fft
Other functions named fft
```

*fx* >>



# 1. Einführung – 1.2 Benutzeroberfläche: MATLAB Hilfe



# 1. Einführung – 1.2 Benutzeroberfläche

## Tastursteuerung:

Schnellere Eingabe von alten Befehlen im Command-Window:

ctrl-P (Previous) und ctrl-N (für Next)  
oder **Cursor-Tasten**

Weitere:

- ctrl-P In der Command History nach oben springen
- ctrl-N In der Command History nach unten springen
- ctrl-B Eine Stelle nach links springen
- ctrl-F Eine Stelle nach rechts springen
- ctrl-A An den Zeilenanfang springen
- ctrl-E An das Zeilenende springen
- ctrl-U Eine Zeile löschen
- ctrl-C Laufende Rechnung stoppen



# 1. Einführung – 1.2 Benutzeroberfläche

## Demos / Erste Schritte:

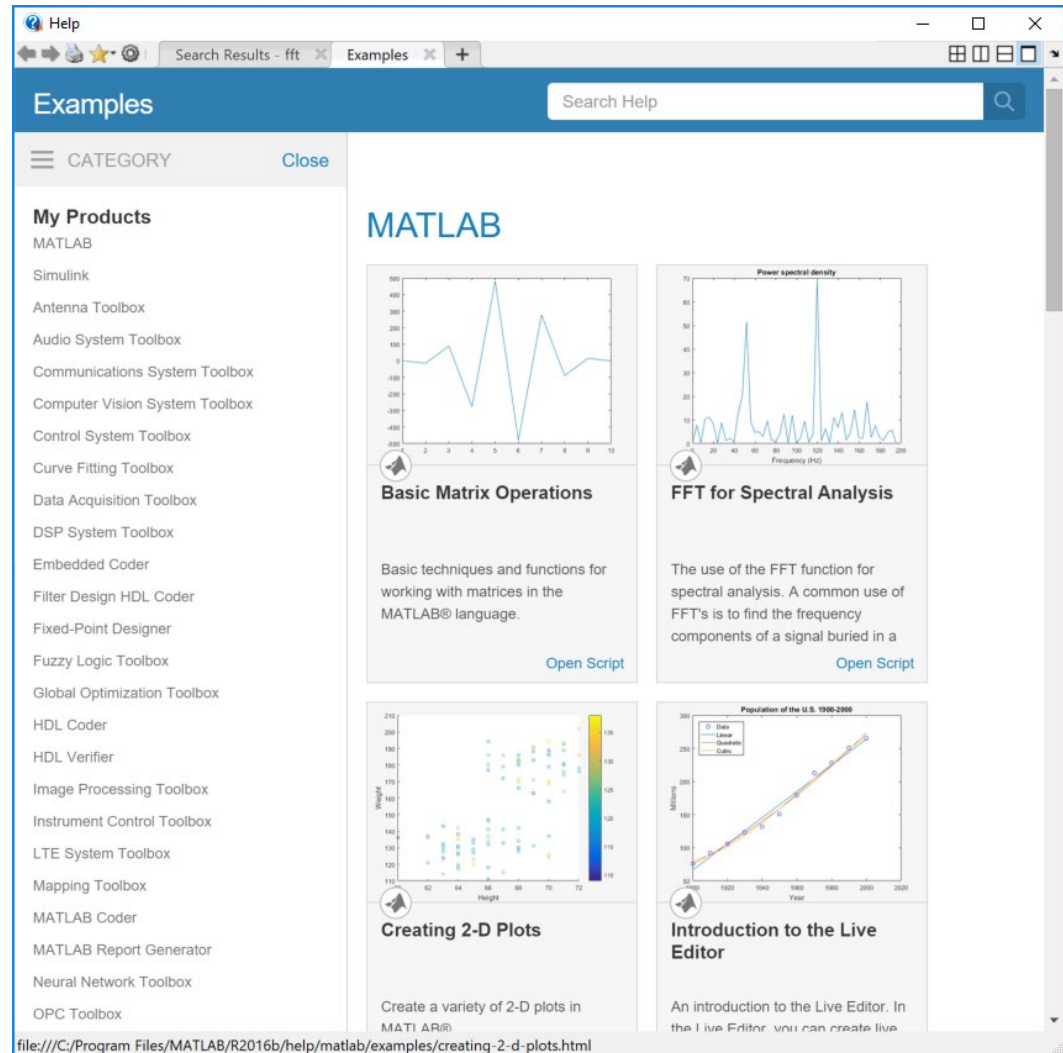
Versuchen Sie

>> why

>> spy

oder starten Sie doch  
besser eine Demo.

>> demo



# 1. Einführung – 1.3 Der Matlab-Workspace

## Der Matlab-Workspace:

Alle Variablen die direkt in der Kommandozeile oder in einem m-Skript definiert werden, sind allen anderen m-Skripten oder bei Aufrufen in der Kommandozeile (sowie unter Simulink) bekannt. Nur Funktionen haben einen eigenen Variablenbereich.

Vorteil: Einfaches interaktives Arbeiten mit Kommandozeileingabe und m-Skripten.

Nachteil: Häufiger „Datensalat“ → Diszipliniertes Arbeiten notwendig.

---

## Wichtige Befehle für den Workspace:

- |                                             |                                                       |
|---------------------------------------------|-------------------------------------------------------|
| <code>&gt;&gt;clear &lt;variable&gt;</code> | → Löschen der Variablen <i>variable</i> vom Workspace |
| <code>&gt;&gt;clear all</code>              | → Löschen aller Variablen vom Workspace               |
| <code>&gt;&gt;who</code>                    | → Anzeigen aller Variablen des Workspace              |
| <code>&gt;&gt;whos</code>                   | → Anzeigen aller Variablen mit Zusatzinformation      |



# 1. Einführung – 1.3 Der Matlab-Workspace

```
>> who
```

Your variables are:

```
A  a  b  d  m  n
```

```
>> whos
```

| Name | Size | Bytes | Class  | Attributes |
|------|------|-------|--------|------------|
| A    | 3x5  | 120   | double |            |
| a    | 2x3  | 152   | cell   |            |
| b    | 1x1  | 8     | double |            |
| d    | 1x1  | 16    | double | complex    |
| m    | 1x1  | 4     | uint32 |            |
| n    | 1x1  | 4     | int32  |            |



# 1. Einführung – 1.3 Der Matlab-Workspace

## Zuweisungen einer Variablen:

Wie bei Interpretersprachen üblich, erfolgt keine Deklaration:

...

```
>> my_variable = 5;
```

Das Semikolon ist nicht notwendig, unterdrückt aber die Ausgabe des Wertes, vgl.:

...

```
>> myvariable = 5
```

```
myvariable =
```

```
5
```



# 1. Einführung – 1.3 Der Matlab-Workspace

## Regeln für Variablennamen:

- Unterscheidung zwischen Groß- und Kleinschreibung
  - Der Variablenname muss mit einem Buchstaben beginnen, anschließend könne beliebige Zahlen oder Buchstaben folgen. '\_' ist zugelassen aber keine Sonderzeichen.
  - Matlab berücksichtigt nur die ersten 19 Zeichen eines Variablennamens!!!
- 

Bei direkten Berechnungen erfolgt eine Zuweisung an die Standardvariable `ans` (Answer).

...

```
>> 5+2
```

```
ans =
```

```
7
```



# 1. Einführung – 1.4 Vektoren und Matrizen

## Array:

- Ein Komma oder ein Leerzeichen (!) bewirkt ein Trennung der Zeilenelemente

```
>> x = [2, 4, 5];      → Zeilenvektor
```

entspricht

```
>> x = [2 4 5];
```

nicht aber

```
>> x = [245];
```

das entspricht

```
>> x = 245;
```

- Skalare sind für Matlab 1x1-Arrays → „In Matlab ist alles Matrix“
- Komma-Schreibweise zur Trennung ist zu bevorzugen.





# 1. Einführung – 1.4 Vektoren und Matrizen

- Ein Semikolon bewirkt eine Trennung der Spaltenelemente

```
>> y = [2; 4; 5];
```

→ Spaltenvektor

```
>> A = [[1, 2]; [3, 4]]
```

→ 2x2-Matrix

A =

|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

entspricht

```
>> A = [1, 2; 3, 4];
```

```
>> A = [[1; 3], [2; 4]];
```

Nicht möglich:

```
>> A = [1; 3, 2; 4]
```

*??? Error using ==> vertcat*

*CAT arguments dimensions are not consistent.*



# 1. Einführung – 1.4 Vektoren und Matrizen

## Arbeiten mit Arrays:

- Verknüpfen von Arrays:

```
>> a = [1, 2];
```

```
>> b = [3, 4];
```

```
>> A = [a; b]
```

A =

|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

```
>> B = [a, b]
```

B =

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
|---|---|---|---|



# 1. Einführung – 1.4 Vektoren und Matrizen

```
>> c = [5, 6, 7, 8];
```

```
>> C = [a; c]
```

```
??? Error using ==> vertcat
```

```
CAT arguments dimensions are not consistent.
```

```
>> D = [a, b; c]
```

D =

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| 5 | 6 | 7 | 8 |



# 1. Einführung – 1.4 Vektoren und Matrizen

- Indizieren von Arrays:

```
>> A = [[1,2 3 4]; [5,6,7,8]; [9,10,11,12]; [13,14,15,16]];
```

Klassische Indizierung mit Zeilen- und Spaltenindex in runden Klammern:

```
>> A(1,4)
```

→ Auswahl eines Elements

```
ans =
```

4

**Wie immer gilt:** „Zeilen zuerst, Spalten später!“

Der Doppelpunkt dient (als Schreibweise für ...) zur Indizierung eines Bereichs:

```
>> A(2,2:4)
```

→ Auswahl der Elemente 2 bis 4 der 2-ten Zeile

```
ans =
```

6

7

8



# 1. Einführung – 1.4 Vektoren und Matrizen

>> A(1:3,3)      → Auswahl der Elemente 1 bis 3 der 3-ten Spalte

ans =

3  
7  
11

>> A(1:end,3)      → end indiziert das letzte Element

ans =

3  
7  
11  
15



# 1. Einführung – 1.4 Vektoren und Matrizen

Der Doppelpunkt ohne Bereichsangabe indiziert ganze Zeilen oder Spalten:

```
>> A(3, :)
```

→ Extraktion der Zeile 3

```
ans =
```

```
9      10      11      12
```

```
>> A(:, 4)
```

→ Extraktion der Spalte 4

```
ans =
```

```
4  
8  
12  
16
```



# 1. Einführung – 1.4 Vektoren und Matrizen

Indizierung jedes n-ten Elements mit: (n1: n: n2)

```
>> a = [1, 2, 3, 4, 5, 6];  
>> a(1:2:6)
```

ans =

1        3        5

Auch Zuweisungen sind über Doppelpunktschreibweise möglich:

```
>> b(2:4) = a(3:5)
```

b =

0        3        4        5

```
>> b(1:2:5) = a(1:3)
```

b =

1        3        2        5        3



# 1. Einführung – 1.4 Vektoren und Matrizen

Zusätzlich sind die Elemente auch bei Matrizen „linear“ indiziert:

```
>> A(6)
```

```
ans =
```

```
6
```



# 1. Einführung – 1.4 Vektoren und Matrizen

```
>> A(1:8)
```

```
ans =
```

```
1      5      9     13      2      6     10     14
```

Merke: Die lineare Indizierung erfolgt Spaltenweise!

```
A =
```

|   |    |    |    |    |    |    |    |    |          |
|---|----|----|----|----|----|----|----|----|----------|
| 1 | 17 | 6  | 24 | 11 | 1  | 16 | 8  | 21 | 15       |
| 2 | 23 | 7  | 5  | 12 | 7  | 17 | 14 | 22 | 16       |
| 3 | 4  | 8  | 6  | 13 | 13 | 18 | 20 | 23 | 22       |
| 4 | 10 | 9  | 12 | 14 | 19 | 19 | 21 | 24 | 3        |
| 5 | 11 | 10 | 18 | 15 | 25 | 20 | 2  | 25 | <b>9</b> |

A

# 1. Einführung – 1.4 Vektoren und Matrizen

- Zuweisungen über Indizierung

```
>> clear A;
```

```
>> A = 1
```

A =

1

```
>> A(3,4) = 2
```

A =

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 2 |

# 1. Einführung – 1.4 Vektoren und Matrizen

- Elementare Operationen

```
>> A = [1, 2; 3, 4];
```

```
>> B = [5, 6; 7, 8];
```

```
>> A + B
```

```
ans =
```

```
     6     8  
    10    12
```

```
>> A * B
```

```
ans =
```

```
    19    22  
    43    50
```

```
>> C = [1, 2, 3; 4, 5, 6];
```

```
>> A + C
```

```
??? Error using ==> plus  
Matrix dimensions must agree.
```



# 1. Einführung – 1.4 Vektoren und Matrizen

```
>> 2 * A
```

```
ans =
```

```
    2    4  
    6    8
```

```
>> b = [2; 4];
```

```
>> A * b
```

```
ans =
```

```
    10  
    22
```

```
>> c = [2, 5];
```

```
>> c * A
```

```
ans =
```

```
    17    24
```



# 1. Einführung – 1.4 Vektoren und Matrizen

```
>> c * b
```

```
ans =
```

```
24
```

```
>> b * c
```

```
ans =
```

```
4    10  
8    20
```

```
>> d = [1; 2; 3;];
```

```
>> e = [3; 2; 1;];
```

```
>> cross(d,e)
```

```
ans =
```

```
-4  
8  
-4
```

```
>> dot(d,e)
```

```
ans =
```

```
10
```



# 1. Einführung – 1.4 Vektoren und Matrizen

Es funktionieren auch:

```
>> A + 3
```

```
ans =
```

|   |   |
|---|---|
| 4 | 5 |
| 6 | 7 |



# 1. Einführung – 1.4 Vektoren und Matrizen

Elementweise Operationen (bei Multiplikation, Division, Potenzen) werden durch einen Punkt vor dem Operator erzwungen

```
>> A .* B
```

```
ans =
```

```
    5    12  
   21    32
```

```
>> A ./ B
```

```
ans =
```

```
   0.2000   0.3333  
   0.4286   0.5000
```

```
>> A.^2
```

→ entspricht `power(A, 2)`

```
ans =
```

```
    1     4  
    9    16
```



# 1. Einführung – 1.4 Vektoren und Matrizen

>>  $A^2$                        $\rightarrow$  entspricht    `mpower(A, 2)`

ans =

|    |    |
|----|----|
| 7  | 10 |
| 15 | 22 |





# 1. Einführung – 1.4 Vektoren und Matrizen

Lösen eines linearen Gleichungssystems:

$$x_1 + x_2 - x_3 = 0$$

$$2x_1 + x_2 + x_3 = 1$$

$$x_1 - 3x_3 = -1$$

wiedergegeben werden als

$$\underbrace{\begin{pmatrix} 1 & 1 & -1 \\ 2 & 1 & 1 \\ 1 & 0 & -3 \end{pmatrix}}_A \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}}_x = \underbrace{\begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix}}_b$$

```
>> A = [1, 1, -1; 2, 1, 1; 1, 0, -3];
```

```
>> b = [0; 1; -1];
```

```
>> x = inv(A) * b
```

x =

0.2000

0.2000

0.4000



# 1. Einführung – 1.4 Vektoren und Matrizen

```
>> x = A \ b
```

```
x =
```

```
0.2000
```

```
0.2000
```

```
0.4000
```

anders:

```
>> b = [0, 1, -1];
```

```
>> x = b / A
```

```
x =
```

```
1.6000    -0.6000    -0.4000
```

'\'  
'/'

→ Multiplikation der Inversen von Links

→ Multiplikation der Inversen von Rechts



# 1. Einführung – 1.4 Vektoren und Matrizen

## Befehle zur Umformung von Matrizen und Arrays:

```
>> A = [1, 1, -1; 2, 1, 1; 1, 0, -3]
```

A =

|   |   |    |
|---|---|----|
| 1 | 1 | -1 |
| 2 | 1 | 1  |
| 1 | 0 | -3 |

```
>> B = A'           → entspricht   B = transpose(A)
```

B =

|    |   |    |
|----|---|----|
| 1  | 2 | 1  |
| 1  | 1 | 0  |
| -1 | 1 | -3 |

Unterscheidung:  $A.'$  → Transponieren ohne Konjugieren



# 1. Einführung – 1.4 Vektoren und Matrizen

```
>> size(A)
```

```
ans =
```

```
      3      3
```

```
>> length(b)
```

```
ans =
```

```
      3
```

```
>> reshape(A,1,9)
```

```
ans =
```

```
      1      2      1      1      1      0     -1      1     -3
```

→ Siehe auch `flipud` und `fliplr`



# 1. Einführung – 1.4 Vektoren und Matrizen

## Vektorisierung:

Die meisten Operatoren und Funktionen (zumindest die elementaren mathematische Funktionen sind „vektorisert“, d. h. können direkt auf Vektoren und Matrizen angewendet werden. Die Operationen werden außer bei speziellen Operationen element-weise ausgeführt:

```
>> x = linspace(0, 2*pi, 100);  
>> y = sin(x) + sin(2*x);  
>> plot(x,y);
```

```
>> max(y)
```

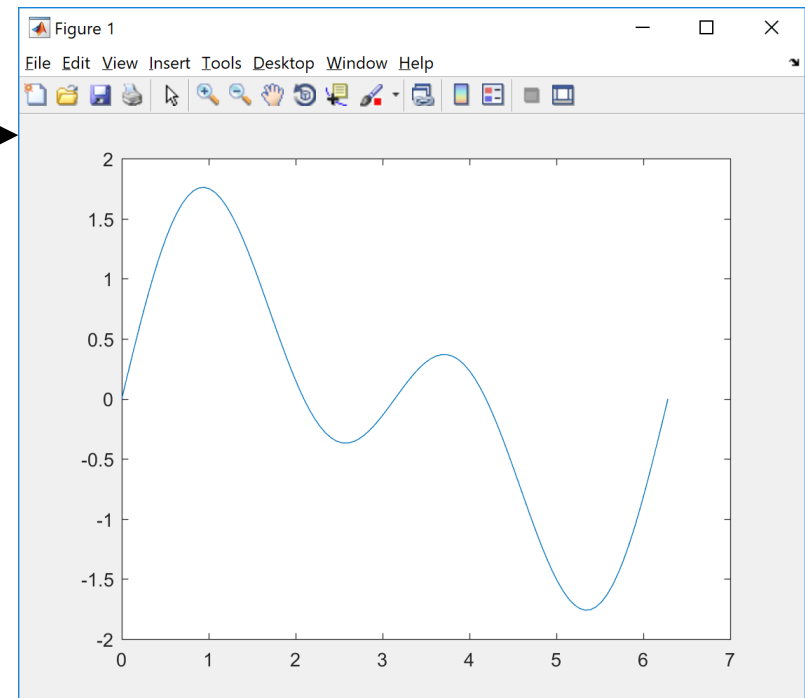
```
ans =
```

```
1.7596
```

```
>> mean(y)
```

```
ans =
```

```
-9.0615e-017
```



# 1. Einführung – 1.5 M-Skripte und M-Editor

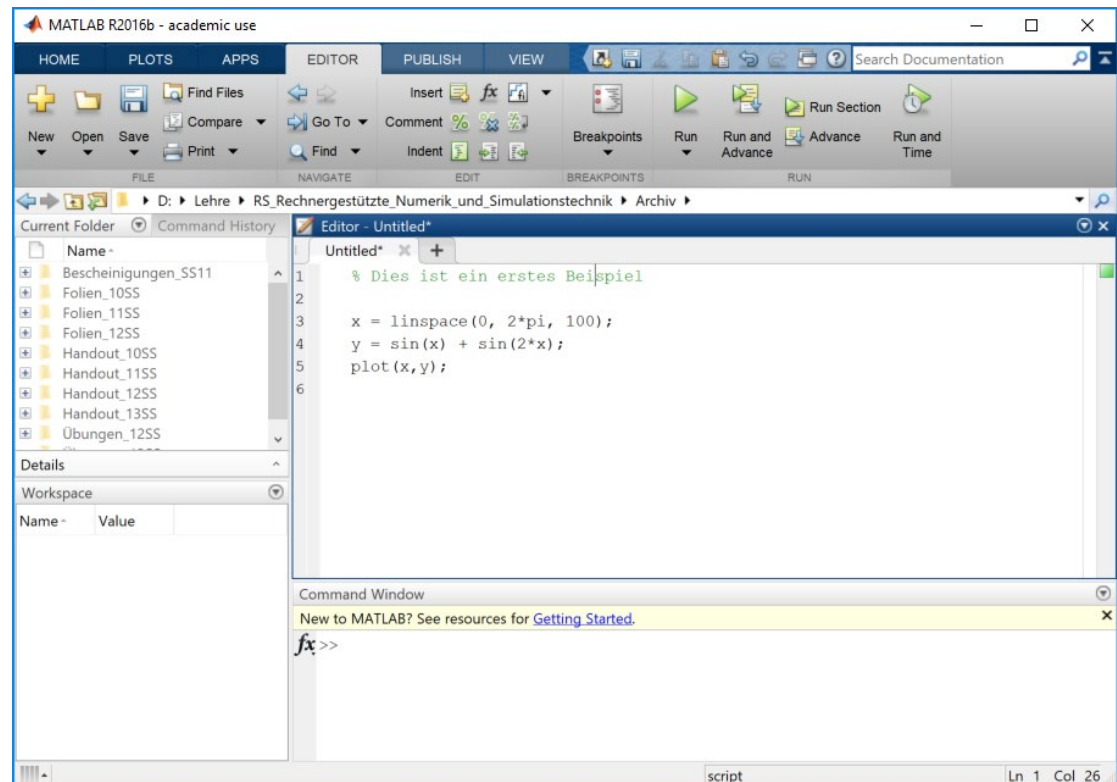
## M-Skripte sind ...

- Text-Dateien von sequentiell ausführbaren Matlab-Befehlen die theoretische auch in der Kommandozeile hätten ausgeführt werden können.

Bsp:

Der Skriptaufrufe erfolgt durch

- <myscript> in der Kommandozeile.
- Grünen „Run“-Button



# 1. Einführung – 1.5 M-Skripte und M-Editor

## Elementares zur Verwendung in m-Skripten:

- % → Führt einen Kommentar an (Rest der Zeile)
- %% → Markiert einen Zellenanfang
- ... → Setzt den Befehl in der nächsten Zeile fort

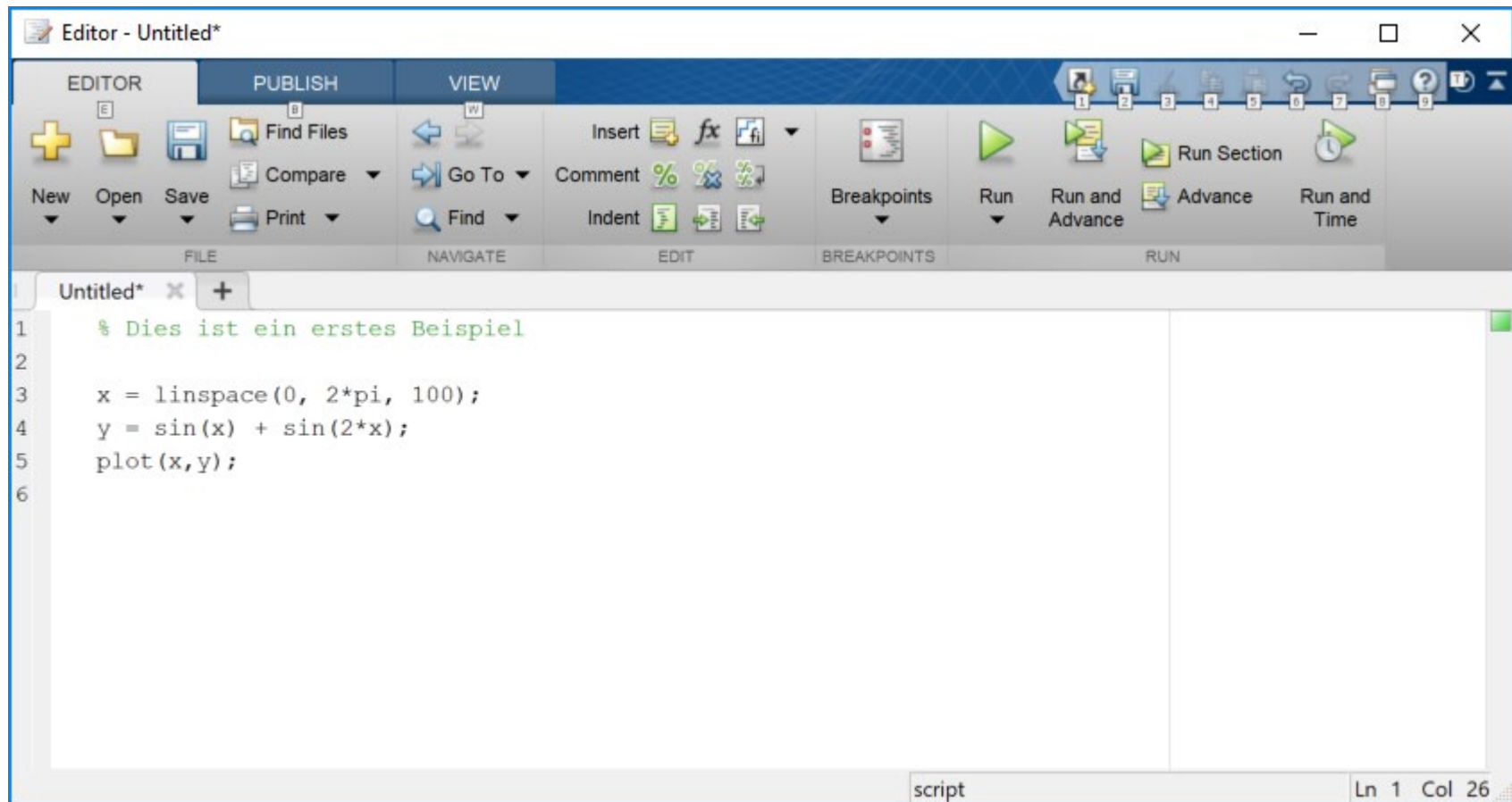
Bsp.

```
A = [1, 0, 3, 5; ...  
     3, 1, 2, 4; ...  
     6, 5, 7, 8; ...  
     4, 2, 6, 8];
```



# 1. Einführung – 1.5 M-Skripte und M-Editor

## Der Matlab-Editor:





# 1. Einführung – 1.5 M-Skripte und M-Editor

## Eigenschaften des Matlab-Editor:

- Aufruf über Doppelklick auf eine m-Datei im Explorer oder Matlab oder
- über den Befehl

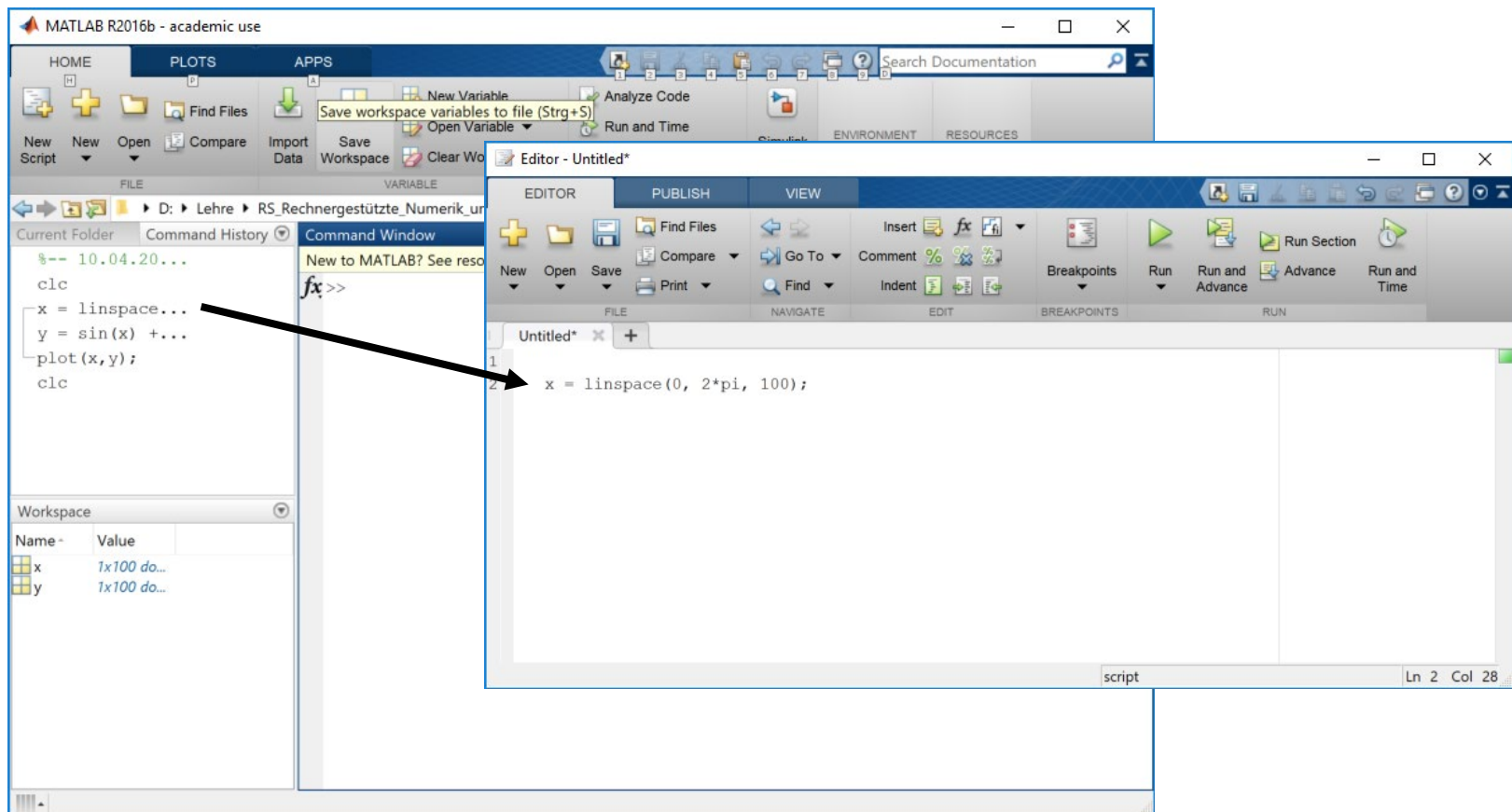
```
>> edit <dateiname>
```

- Spezielles *Highlighting* für die Matlab-Syntax
- Strukturierung und Einrückhilfen (‘on-the-fly’ oder *Control-I*)
- Debug-Funktionen
- Strukturiertes Arbeiten mit Zellen
- Veröffentlichungsfunktionen



# 1. Einführung – 1.5 M-Skripte und M-Editor

Kommandos können per „Drag and Drop“  
aus der Command History in den Editor  
kopiert werden



# 1. Einführung – 1.6 Kontrollstrukturen

## Kontrollstrukturen in m-Skripten und m-Funktionen:

- If-Anweisung
- Switch-Case-Anweisung
- For-Schleife (Zählschleife)
- While-Schleife (Schleife mit Bedingung)
- Break-Anweisung
- Try-Catch-Anweisung



# 1. Einführung – 1.6 Kontrollstrukturen

## If-Anweisung:

```
if (<Bedingung 1>
    <Anweisungen 1>
elseif (<Bedingung 2>)
    <Anweisungen 2>
else
    <Anweisungen 3>
end
```

- Blau-markiertes ist optional !
- Wichtig: `end` schließt die bei Matlab Kontrollstrukturen und Funktionen ab
- Bedingungen sind boolsche Ausdrücke  
-> Zur Erinnerung: Numerische Werte ungleich „null“ werden als wahr interpretiert.



# 1. Einführung – 1.6 Kontrollstrukturen

## Beispiele:

```
if a > 2
    b = 3;
else
    b = 2;
end
```

```
if floor(x)
    disp('x ist größer <?>');
end
```

→ Was ist <?> ?

```
if isnumeric(b)
    disp(['b = ', num2str(b)]);
elseif isstr(b)
    disp(['b = ', b]);
else
    error('Falsches Format für b');
end
```



# 1. Einführung – 1.6 Kontrollstrukturen

## Switch-Case-Anweisung:

```
switch (<Ausdruck>
  case <Konstante1>:
    <Anweisung1>
  case <Konstante2>:
    <Anweisung2>
  otherwise
    <Anweisung3>
end
```

- Blau-markiertes ist optional !
- Beachte den Doppelpunkt



# 1. Einführung – 1.6 Kontrollstrukturen

## Beispiele:

```
switch Richtung
case 'rechts':
    disp('Bitte fahren Sie nach rechts');
case 'links':
    disp('Bitte fahren Sie nach links');
otherwise
    disp('Bitte fahren Sie geradeaus');
End
```

```
>> Richtung = 'rechts';
>> myscript
```

```
Bitte fahren Sie nach rechts
```



# 1. Einführung – 1.6 Kontrollstrukturen

## For-Schleife (Zählschleife):

```
for <Laufindex>=<Initialisierung>:<Schrittweite>:<Endwert>  
    <Anweisung>  
end
```

## Beispiele:

```
b = 3;  
a = 0;
```

```
for index = 1:1:20  
    a=a+b;  
end
```

```
disp(a);    → ???
```





# 1. Einführung – 1.6 Kontrollstrukturen

## While-Schleife:

```
while(<Bedingung>)  
    <Anweisung>  
end
```

## Beispiele:

```
a = 25;  
b = 3;  
c = -1;
```

```
while(a > 0)  
    a = a - b;  
    c = c + 1;  
end
```

```
disp(a);    → ???
```



# 1. Einführung – 1.6 Kontrollstrukturen

## Break-Anweisung :

Unterbrechung einer `for`- oder `while`-Schleife:

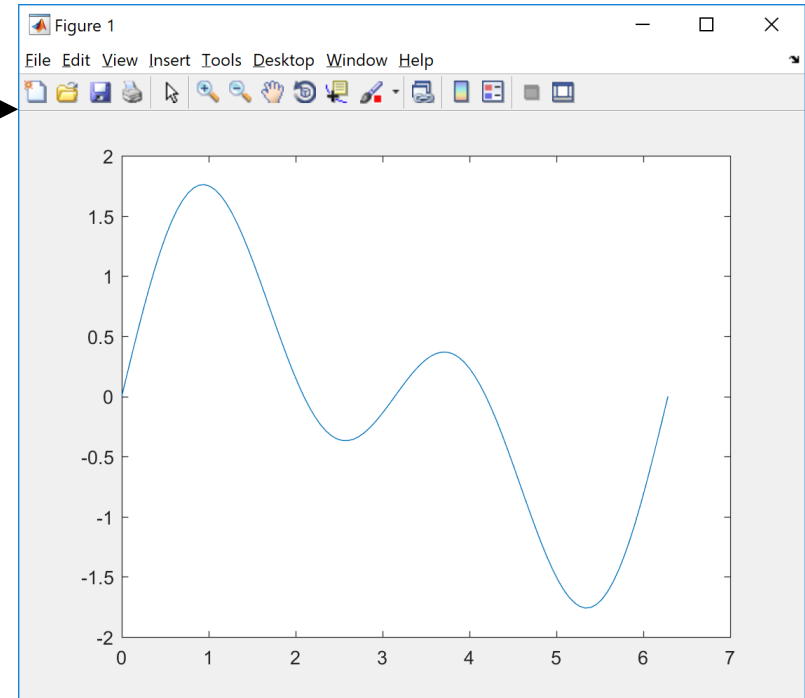
```
b = 3;  
a = 0;  
  
for index = 1:1:20  
    a=a+b;  
    if a > 15  
        disp('zu groß');  
        break;  
    end  
end  
  
disp(a);
```



# 1. Einführung – 1.7 Grafik

## Plot-Befehl:

```
>> x = linspace(0, 2*pi, 100);  
>> y = sin(x) + sin(2*x);  
>> plot(x,y);
```



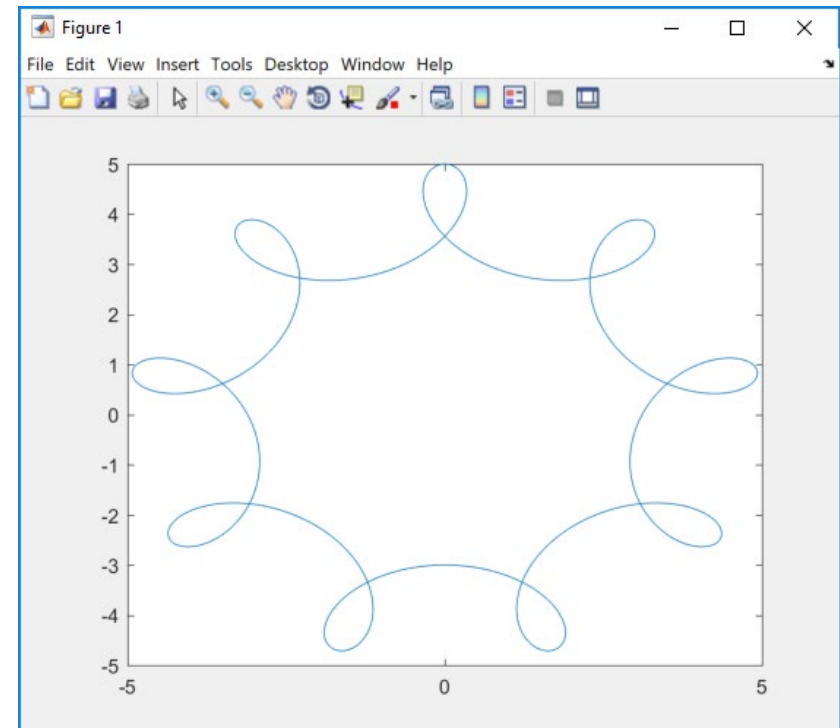
# 1. Einführung – 1.7 Grafik

## Wichtig:

Der bekannte `plot`-Befehl kann (wie auch die anderen Befehle für „ebene Grafiken“) nicht nur Graphen im mathematischen Sinne, sondern beliebige *Polygonzüge* zeichnen, die durch die Eingangsvektoren definiert sind.

## Beispiel:

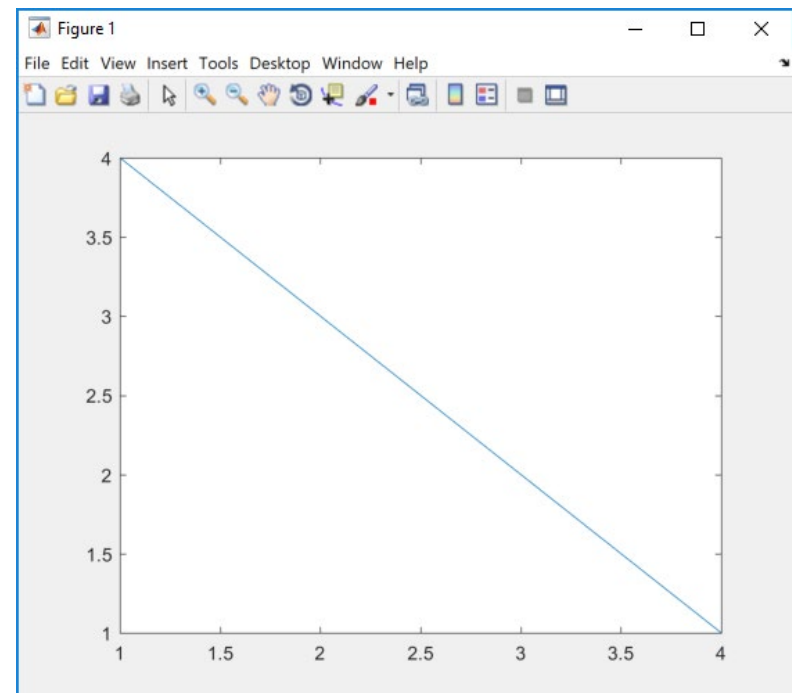
```
>> t=linspace(0,2*pi,1000);  
>> x=4*cos(t)+sin(8*t);  
>> y=4*sin(t)+cos(8*t);  
>> plot(x,y)
```



# 1. Einführung – 1.7 Grafik

Wird bei `plot` nur ein Vektor angegeben, dann wird als x-Achse automatisch ein Vektor mit von 1 aufsteigenden Werten verwendet:

```
>> plot([4,3,2,1]);
```



# 1. Einführung – 1.7 Grafik

## Linienformatierung:

Die Grafikbefehle für ebene Grafiken können Formatierungsanweisungen enthalten:

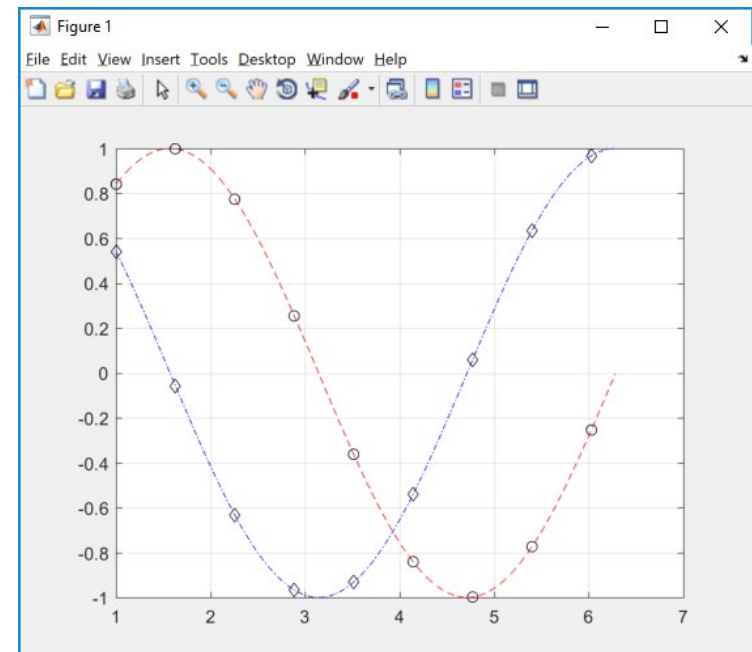
```
>> plot(x,y,'<color><style>');
```



Farbauswahl      Linienart bzw. Markierungsart

### Beispiel:

```
>> plot((1:0.01:2*pi),sin(1:0.01:2*pi),'r--')  
>> hold on;  
>> grid on;  
>> plot((1:2*pi/10:2*pi),sin(1:2*pi/10:2*pi),'ko')  
>> plot((1:0.01:2*pi),cos(1:0.01:2*pi),'b-.')  
>> plot((1:2*pi/10:2*pi),cos(1:2*pi/10:2*pi),'kd')
```



# 1. Einführung – 1.7 Grafik

## Schlüssel für Linienformatierung:

Farbe:

|   |           |
|---|-----------|
| b | → Blau    |
| g | → Grün    |
| r | → Rot     |
| c | → Cyan    |
| m | → Magenta |
| y | → Gelb    |
| k | → Schwarz |

Markierungen:

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| . | • | o | ○ | x | × | + | + | * | * |
| s | □ | d | ◇ | v | ▽ | ^ | △ | < | ◁ |
| > | ▷ | p | ☆ | h | ☆ |   |   |   |   |

Linien:

-. -.-.-.- -- -

